Scenario

As a security analyst, it's important to know how to capture and filter network traffic in a Linux environment. You'll also need to know the basic concepts associated with network interfaces.

In this lab activity, you'll perform tasks associated with using tcpdump to capture network traffic. You'll capture the data in a packet capture (p-cap) file and then examine the contents of the captured packet data to focus on specific types of traffic.

You're a network analyst who needs to use tcpdump to capture and analyze live network traffic from a Linux virtual machine.

Task 1. Identify network interfaces

In this task, you must identify the network interfaces that can be used to capture network packet data.

```
analyst@6011a3429aa5:~$ sudo ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1460
        inet 172.17.0.2  netmask 255.255.0.0  broadcast 172.17.255.255
        ether 02:42:ac:11:00:02  txqueuelen 0  (Ethernet)
        RX packets 850  bytes 14799169 (14.1 MiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 561  bytes 46826 (45.7 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 55  bytes 8753 (8.5 KiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 55  bytes 8753 (8.5 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

The Ethernet network interface is identified by the entry with the eth prefix. So in this lab we will use `eth0` as the interface from where we will capture network packet data in the following tasks.

```
analyst@6011a3429aa5:~$ sudo tcpdump -D
1.eth0 [Up, Running, Connected]
2.any (Pseudo-device that captures on all interfaces) [Up, Running]
3.lo [Up, Running, Loopback]
4.bluetooth-monitor (Bluetooth Linux Monitor) [Wireless]
5.nflog (Linux netfilter log (NFLOG) interface) [none]
6.nfqueue (Linux netfilter queue (NFQUEUE) interface) [none]
7.dbus-system (D-Bus system bus) [none]
8.dbus-session (D-Bus session bus) [none]
```

Using the command `sudo tcpdump -D` will allow us to identify which network interfaces are available. This may be useful on systems that do not include the ifconfig command.

Task 2. Inspect the network traffic of a network interface with tcpdump

In this task, you must use tcpdump to filter live network packet traffic on an interface.

```
analyst@6011a3429aa5:~$ sudo tcpdump -i eth0 -v -c5
tcpdump: listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
15:05:13.881772 IP (tos 0x0, ttl 64, id 62295, offset 0, flags [DF], proto TCP (6), length 59)
    6011a3429aa5.5000 > nginx-us-central1-c.c.qwiklabs-terminal-vms-prod-00.internal.57886: Flags [P.], cksum 0x5890 (incorrect -> 0x66ed), seq 3711828011:3711828018, ack 4244768068, win 498, options [nop,nop,TS val 1156887920 ecr 2770835176], length 7
15:05:13.881871 IP (tos 0x0, ttl 64, id 11120, offset 0, flags [DF], proto TCP (6), length 139)
    6011a3429aa5.5000 > nginx-us-central1-c.c.qwiklabs-terminal-vms-prod-00.internal.35772: Flags [P.], cksum 0x58e0 (incorrect -> 0x9ca7), seq 1721409309:1721409396, ack 2043128000, win 498, options [nop,nop,TS val 1156887920 ecr 2770835176], length 87
15:05:13.882043 IP (tos 0x0, ttl 63, id 48596, offset 0, flags [DF], proto TCP (6), length 52)
    nginx-us-central1-c.c.qwiklabs-terminal-vms-prod-00.internal.57886 > 6011a3429aa5.5000: Flags [.], cksum 0x1084 (correct), ack 7, win 504, options [nop,nop,TS val 2770835187 ecr 1156887920], length 0
15:05:13.882044 IP (tos 0x0, ttl 63, id 25331, offset 0, flags [DF], proto TCP (6), length 52)
    nginx-us-central1-c.c.qwiklabs-terminal-vms-prod-00.internal.35772 > 6011a3429aa5.5000: Flags [.], cksum 0x1e01 (correct), ack 87, win 510, options [nop,nop,TS val 2770835187 ecr 1156887920], length 0
15:05:13.981795 IP (tos 0x0, ttl 64, id 22306, offset 0, flags [DF], proto UDP (17), length 70)
    6011a3429aa5.42223 > metadata.google.internal.domain: 64718+ PTR? 59.0.20.172.in-addr.arpa. (42)
5 packets captured
20 packets received by filter
0 packets dropped by kernel
```

sudo tcpdump -i eth0 -v -c5

This command will run tcpdump with the following options:

- -i eth0: Capture data specifically from the eth0 interface.
- -v: Display detailed packet data.
- -c5: Capture 5 packets of data.

Task 3. Capture network traffic with tcpdump

In this task, you will use tcpdump to save the captured network data to a packet capture file.

In the previous command, you used tcpdump to stream all network traffic. Here, you will use a filter and other tcpdump configuration options to save a small sample that contains only web (TCP port 80) network packet data.

```
analyst@6011a3429aa5:~$ sudo tcpdump -i eth0 -nn -c9 port 80 -w capture.pcap &
[1] 656
analyst@6011a3429aa5:~$ tcpdump: listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
```

Here I used the command sudo tcpdump -i eth0 -nn -c9 port 80 -w capture.pcap & to capture packet data into a file called capture.pcap

This command will run tcpdump in the background with the following options:

- -i eth0: Capture data from the eth0 interface.
- -nn: Do not attempt to resolve IP addresses or ports to names.This is best practice from a security perspective, as the lookup data may not be valid. It also prevents malicious actors from being alerted to an investigation.
- -c9: Capture 9 packets of data and then exit.
- port 80: Filter only port 80 traffic. This is the default HTTP port.
- -w capture.pcap: Save the captured data to the named file.
- &: This is an instruction to the Bash shell to run the command in the background.

```
analyst@6011a3429aa5:~$ curl opensource.google.com
<HTML><HEAD><meta http-equiv="content-type" content="text/html;charset=utf-8">
<TITLE>301 Moved</TITLE></HEAD><BODY>
<H1>301 Moved</H1>
The document has moved
<A HREF="https://opensource.google/">here</A>.
</BODY></HTML>
analyst@6011a3429aa5:~$ 9 packets captured
10 packets received by filter
0 packets dropped by kernel
```

Here I user the command curl opensource.google.com to generate some HTTP (port 80) traffic.

Task 4. Filter the captured packet data

In this task, use tcpdump to filter data from the packet capture file you saved previously.

```
analyst@6011a3429aa5:~$ sudo tcpdump -nn -r capture.pcap -v
reading from file capture.pcap, link-type EN10MB (Ethernet), snapshot length 262144
15:11:50.653880 IP (tos 0x0, ttl 64, id 43642, offset 0, flags [DF], proto TCP (6), length 60)
    172.17.0.2.55482 > 192.179.17.100.80: Flags [S], cksum 0x7e59 (incorrect -> 0x2677), seq 3814116893, win 65320, options [mss 1420,sackOK,TS val 1324534507 ecr 0,nop,wscale 7], length 0
15:11:50.655051 IP (tos 0x0, ttl 126, id 0, offset 0, flags [DF], proto TCP (6), length 60)
    192.179.17.100.80 > 172.17.0.2.55482: Flags [S.], cksum 0x4176 (correct), seq 3131037579, ack 3814116894, win 65535, options [mss 1420,sackOK,TS val 3217855007 ecr 1324534507,nop,wscale 8], length 0
15:11:50.655095 IP (tos 0x0, ttl 64, id 43643, offset 0, flags [DF], proto TCP (6), length 52)
    172.17.0.2.55482 > 192.179.17.100.80: Flags [.], cksum 0x7e51 (incorrect -> 0x6e1a), ack 1, win 511, options [nop,nop,TS val 1324534509 ecr 3217855007], length 0
15:11:50.655193 IP (tos 0x0, ttl 64, id 43644, offset 0, flags [DF], proto TCP (6), length 137)
    172.17.0.2.55482 > 192.179.17.100.80: Flags [P.], cksum 0x7ea6 (incorrect -> 0xdac8), seq 1:86, ack 1, win 511, options [nop,nop,TS val 1324534509 ecr 3217855007], length 85: HTTP, length: 85
        GET / HTTP/1.1
        Host: opensource.google.com
        User-Agent: curl/7.88.1
        Accept: */*

15:11:50.655333 IP (tos 0x0, ttl 126, id 0, offset 0, flags [DF], proto TCP (6), length 52)
    192.179.17.100.80 > 172.17.0.2.55482: Flags [.], cksum 0x6ba8 (correct), ack 86, win 1051, options [nop,nop,TS val 3217855008 ecr 1324534509], length 0
15:11:50.662949 IP (tos 0x0, ttl 126, id 0, offset 0, flags [DF], proto TCP (6), length 590)
    192.179.17.100.80 > 172.17.0.2.55482: Flags [P.], cksum 0x3eb0 (correct), seq 1:539, ack 86, win 1051, options [nop,nop,TS val 3217855015 ecr 1324534509], length 538: HTTP, length: 538
        HTTP/1.1 301 Moved Permanently
        Location: https://opensource.google/
        Content-Type: text/html; charset=UTF-8
        X-Content-Type-Options: nosniff
        Date: Fri, 23 Oct 2026 15:11:50 GMT
        Expires: Fri, 23 Oct 2026 15:41:50 GMT
        Cache-Control: public, max-age=1800
        Server: sffe
        Content-Length: 223
        X-XSS-Protection: 0

        <HTML><HEAD><meta http-equiv="content-type" content="text/html;charset=utf-8">
        <TITLE>301 Moved</TITLE></HEAD><BODY>
        <H1>301 Moved</H1>
        The document has moved
        <A HREF="https://opensource.google/">here</A>.
        </BODY></HTML>
15:11:50.662963 IP (tos 0x0, ttl 64, id 43645, offset 0, flags [DF], proto TCP (6), length 52)
    172.17.0.2.55482 > 192.179.17.100.80: Flags [.], cksum 0x7e51 (incorrect -> 0x6ba0), ack 539, win 507, options [nop,nop,TS val 1324534516 ecr 3217855015], length 0
15:11:50.663448 IP (tos 0x0, ttl 64, id 43646, offset 0, flags [DF], proto TCP (6), length 52)
    172.17.0.2.55482 > 192.179.17.100.80: Flags [F.], cksum 0x7e51 (incorrect -> 0x6b9e), seq 86, ack 539, win 507, options [nop,nop,TS val 1324534517 ecr 3217855015], length 0
```

Here I used the command sudo tcpdump -nn -r capture.pcap -v to filter the packet header data from the capture.pcap capture file.

This command will run tcpdump with the following options:

- -nn: Disable port and protocol name lookup.
- -r: Read capture data from the named file.
- -v: Display detailed packet data.

```
analyst@6011a3429aa5:~$ sudo tcpdump -nn -r capture.pcap -X
reading from file capture.pcap, link-type EN10MB (Ethernet), snapshot length 262144
15:11:50.653880 IP 172.17.0.2.55482 > 192.179.17.100.80: Flags [S], seq 3814116893, win 65320, options [mss 1420,sackOK,TS val 1324534507 ecr 0,nop,wscale 7], length 0
        0x0000:  4500 003c aa7a 4000 4006 1217 ac11 0002  E..<.z@.@.......
        0x0010:  c0b3 1164 d8ba 0050 e356 ce1d 0000 0000  ...d...P.V......
        0x0020:  a002 ff28 7e59 0000 0204 058c 0402 080a  ...(~Y..........
        0x0030:  4ef2 caeb 0000 0000 0103 0307            N...........
15:11:50.655051 IP 192.179.17.100.80 > 172.17.0.2.55482: Flags [S.], seq 3131037579, ack 3814116894, win 65535, options [mss 1420,sackOK,TS val 3217855007 ecr 1324534507,nop,wscale 8], length 0
        0x0000:  4500 003c 0000 4000 7e06 7e91 c0b3 1164  E..<..@.~.~....d
        0x0010:  ac11 0002 0050 d8ba ba9f d78b e356 cele  .....P.......V..
        0x0020:  a012 ffff 4176 0000 0204 058c 0402 080a  ....Av..........
        0x0030:  bfcc 921f 4ef2 caeb 0103 0308            ....N.......
15:11:50.655095 IP 172.17.0.2.55482 > 192.179.17.100.80: Flags [.], ack 1, win 511, options [nop,nop,TS val 1324534509 ecr 3217855007], length 0
        0x0000:  4500 0034 aa7b 4000 4006 121e ac11 0002  E..4.{@.@.......
        0x0010:  c0b3 1164 d8ba 0050 e356 cele ba9f d78c  ...d...P.V......
        0x0020:  8010 01ff 7e51 0000 0101 080a 4ef2 caed  ....~Q......N...
        0x0030:  bfcc 921f                                ....
15:11:50.655193 IP 172.17.0.2.55482 > 192.179.17.100.80: Flags [P.], seq 1:86, ack 1, win 511, options [nop,nop,TS val 1324534509 ecr 3217855007], length 85: HTTP: GET / HTTP/1.1
        0x0000:  4500 0089 aa7c 4000 4006 11c8 ac11 0002  E....|@.@.......
        0x0010:  c0b3 1164 d8ba 0050 e356 cele ba9f d78c  ...d...P.V......
        0x0020:  8018 01ff 7ea6 0000 0101 080a 4ef2 caed  ....~.......N...
        0x0030:  bfcc 921f 4745 5420 2f20 4854 5450 2f31  ....GET./.HTTP/1
        0x0040:  2e31 0d0a 486f 7374 3a20 6f70 656e 736f  .1..Host:.openso
        0x0050:  7572 6365 2e67 6f6f 676c 652e 636f 6d0d  urce.google.com.
        0x0060:  0a55 7365 722d 4167 656e 743a 2063 7572  .User-Agent:.cur
        0x0070:  6c2f 372e 3838 2e31 0d0a 4163 6365 7074  l/7.88.1..Accept
        0x0080:  3a20 2a2f 2a0d 0a0d 0a                   :.*/*....
15:11:50.655333 IP 192.179.17.100.80 > 172.17.0.2.55482: Flags [.], ack 86, win 1051, options [nop,nop,TS val 3217855008 ecr 1324534509], length 0
        0x0000:  4500 0034 0000 4000 7e06 7e99 c0b3 1164  E..4..@.~.~....d
        0x0010:  ac11 0002 0050 d8ba ba9f d78c e356 ce73  .....P.......V.s
        0x0020:  8010 041b 6ba8 0000 0101 080a bfcc 9220  ....k..........
        0x0030:  4ef2 caed                                N...
15:11:50.662949 IP 192.179.17.100.80 > 172.17.0.2.55482: Flags [P.], seq 1:539, ack 86, win 1051, options [nop,nop,TS val 3217855015 ecr 1324534509], length 538: HTTP: HTTP/1.1 301 Moved Permanently
        0x0000:  4500 024e 0000 4000 7e06 7c7f c0b3 1164  E..N..@.~.|....d
        0x0010:  ac11 0002 0050 d8ba ba9f d78c e356 ce73  .....P.......V.s
        0x0020:  8018 041b 3eb0 0000 0101 080a bfcc 9227  ....>..........'
        0x0030:  4ef2 caed 4854 5450 2f31 2e31 2033 3031  N...HTTP/1.1.301
        0x0040:  204d 6f76 6564 2050 6572 6d61 6e65 6e74  .Moved.Permanent
        0x0050:  6c79 0d0a 4c6f 6361 7469 6f6e 3a20 6874  ly..Location:.ht
        0x0060:  7470 733a 2f2f 6f70 656e 736f 7572 6365  tps://opensource
        0x0070:  2e67 6f6f 676c 652f 0d0a 436f 6e74 656e  .google/..Conten
        0x0080:  742d 5479 7065 3a20 7465 7874 2f68 746d  t-Type:.text/htm
```

Here I used the command sudo tcpdump -nn -r capture.pcap -X to filter the extended packet data from the capture.pcap capture file.

This command will run tcpdump with the following options:

- -nn: Disable port and protocol name lookup.
- -r: Read capture data from the named file.
- -X: Display the hexadecimal and ASCII output format packet data. Security analysts can analyze hexadecimal and ASCII output to detect patterns or anomalies during malware analysis or forensic analysis.